

校园食运通：服务器部署文档

1. 华为云服务器

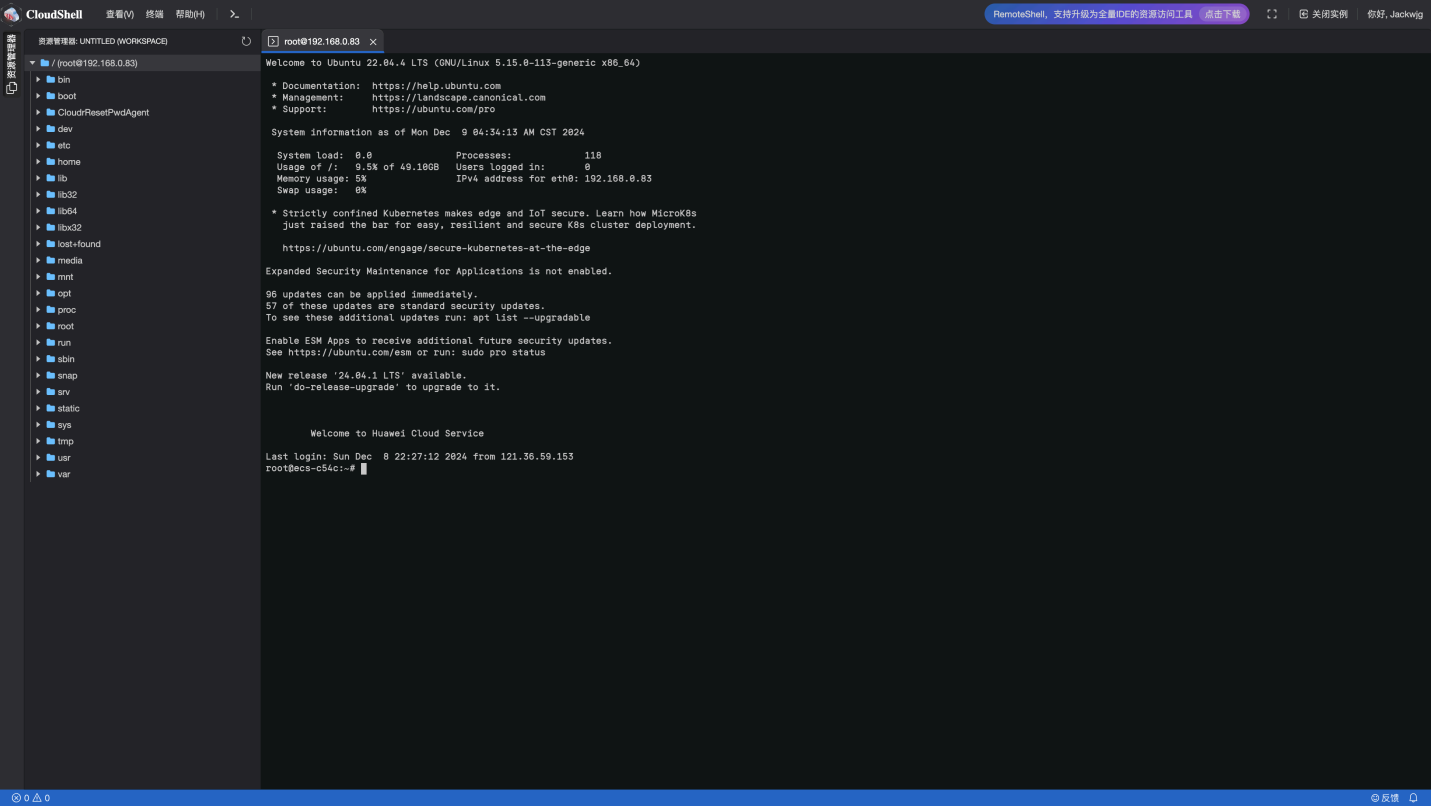
(1) 租借服务器与服务器开机

登陆服务器控制台（<https://console.huaweicloud.com/ecm/?>），选择服务器实例并选择开机。



(2) 登陆服务器运行终端

点击远程登录，然后输入密码，进入服务器终端。



2. 项目配置

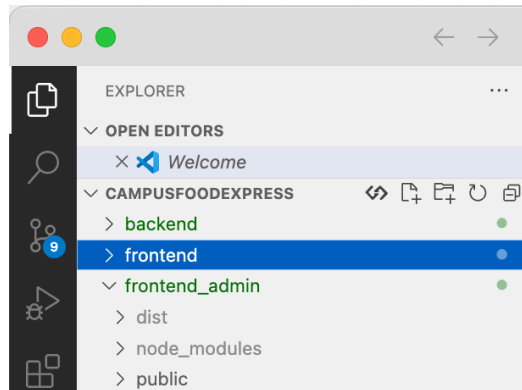
本部分介绍如何部署前端项目在服务器域名（<http://113.44.77.193>）中。在打包项目之后，需要配置 Nginx。Nginx 是一个高性能的开源 Web 服务器和反向代理服务器，广泛用于处理 HTTP 请求、负载均衡和静态内容的高效分发。

(1) 项目前端Vue配置

在项目本地目录输入如下的npm命令完成打包。

```
npm run build
```

执行成功之后会在对应的目录下生成 `dist/` 目录，用于存放项目打包后的代码。



之后使用zip命令打包并上传服务器。

```
zip -r dist.zip dist/
```

上传服务器假设在 `/` 根目录下，使用如下命令修改文件位置并解压。

```
mv /dist.zip /var/www/html  
unzip dist.zip
```

这样项目的前端代码就出现在了其应该出现的位置。

(2) 项目后端配置

在本地项目后端使用zip命令打包。

```
zip -r CampusFoodExpress.zip backend/
```

之后上传服务器并进行解压。

```
mv CampusFoodExpress.zip /var/www  
unzip CampusFoodExpress.zip
```

这样项目后端代码就出现在了其应该出现的位置。需要进入后端文件夹以安装依赖于生产环境和开发环境。

```
cd CampusFoodExpress/  
python -m venv .venv  
source .venv/bin/activate
```

使用 `venv` 虚拟环境中的 `pip` 安装你所需要的依赖，如：

```
pip install flask
```

(3) Nginx安装与配置

使用如下命令安装Nginx。

```
sudo apt update  
sudo apt install nginx
```

然后使用 `nano`（也可以使用 `vim` 等）编辑器，对Nginx的默认配置进行修改。

```
sudo nano /etc/nginx/sites-available/default
```

原有的nginx配置参考如下：

```
server {  
    listen 80 default_server;  
    listen [::]:80 default_server;  
  
    root /var/www/html;  
    index index.html index.htm index.nginx-debian.html;  
  
    server_name _;  
  
    location / {  
        try_files $uri $uri/ =404;  
    }  
}
```

删除原有配置，修改配置如下：

```
server {  
    listen 80;  
    server_name 113.44.77.193;  
  
    # 提供静态文件  
    location /static {  
        root /var/www/CampusFoodExpress;  
    }  
}
```

```
# 默认主页配置
location / {
    root /var/www/html/dist;
    try_files $uri $uri/ index.html;
}

# API 请求代理配置
location /api/ {
    proxy_pass http://127.0.0.1:5000; # 指向 Flask 应用
    proxy_http_version 1.1; # 必须使用 HTTP/1.1, 支持 WebSocket
    proxy_set_header Upgrade $http_upgrade; # 支持 WebSocket 的升级
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header Authorization $http_authorization;
    set $allowed_origin $http_origin; # 动态设置为请求头中的 Origin

    # 添加 CORS 响应头
    add_header Access-Control-Allow-Origin $allowed_origin always;
    add_header Access-Control-Allow-Credentials true always; # 允许凭证 (cookies) 跨域
    add_header Access-Control-Allow-Methods 'GET, POST, PUT, DELETE, OPTIONS' always;
    add_header Access-Control-Allow-Headers 'Content-Type, Authorization' always;

    # 防止超时
    proxy_read_timeout 3600;
    proxy_send_timeout 3600;

    # 处理预检请求 (OPTIONS 请求)
    if ($request_method = 'OPTIONS') {
        add_header Access-Control-Allow-Origin $allowed_origin always;
        add_header Access-Control-Allow-Credentials true always;
        add_header Access-Control-Allow-Methods 'GET, POST, PUT, DELETE, OPTIONS' always;
        add_header Access-Control-Allow-Headers 'Content-Type, Authorization' always;
        return 204; # 返回成功的预检响应
    }
}

location /socket.io/ {
    proxy_pass http://127.0.0.1:5000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    # 不再设置 CORS 头, 由后端处理
}
```

```

        proxy_read_timeout 3600;
        proxy_send_timeout 3600;
    }
    # 配置日志
    access_log /var/log/nginx/access.log;
    error_log /var/log/nginx/error.log;
}

```

需要注意的是，这里提供静态文件的位置为 `/var/www/CampusFoodExpress`，即项目后端存储静态文件的位置。同时，由于本项目同时支持http与socket，所以在配置中对两类请求分别配置了nginx代理，命名为 `api/` 和 `socket.io`。

(4) Nginx相关命令

测试上面的配置是否有误：

```
sudo nginx -t
```

如果配置无误，可以看到下面的输出：

```

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

```

重启nginx命令让上面的配置生效：

```
sudo systemctl restart nginx
```

然后输入下面命令查看nginx状态：

```
sudo systemctl status nginx
```

将会看到如下的输出：

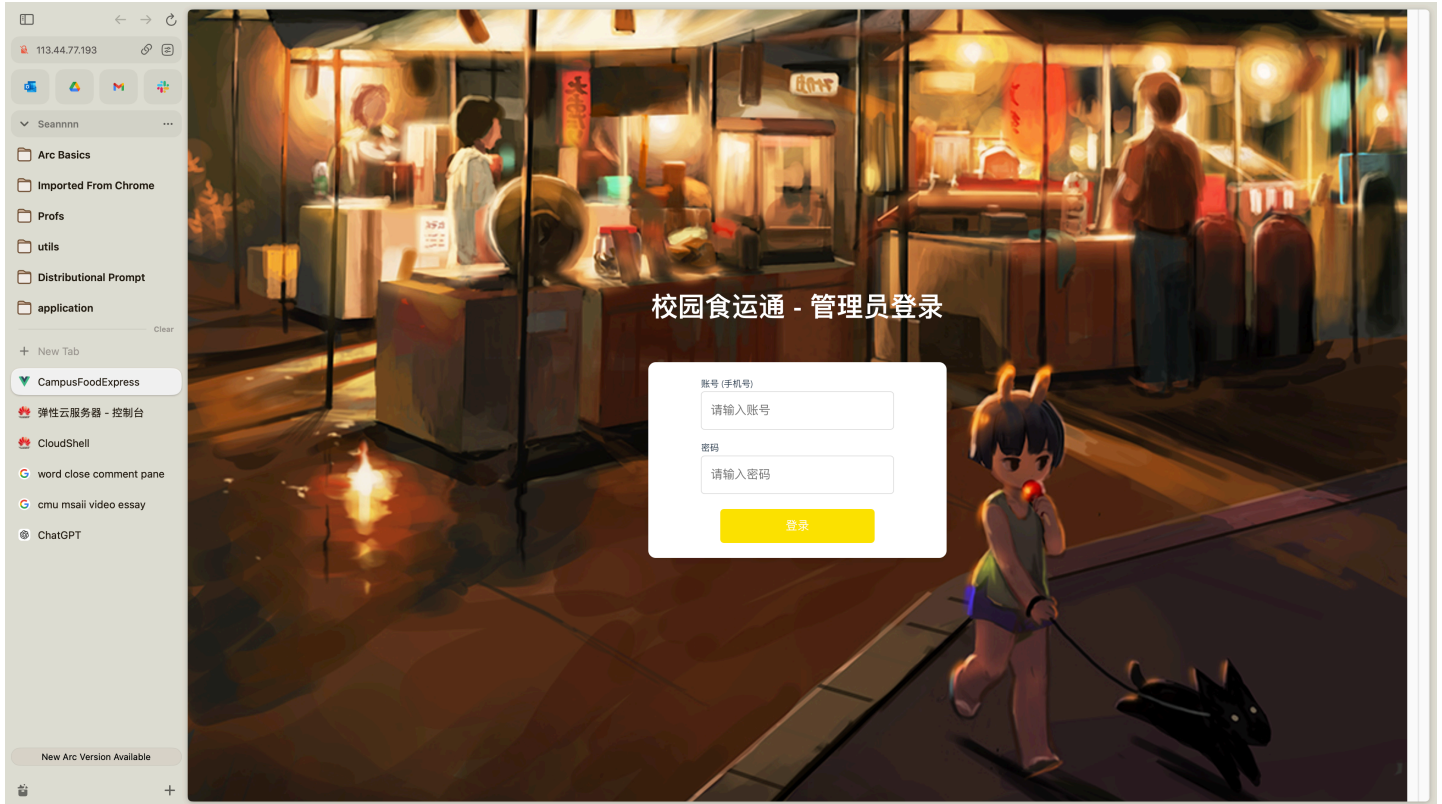
```

root@ecs-c54c:~# sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2024-12-08 05:25:06 CST; 23h ago
     Docs: man:nginx(8)
  Main PID: 64269 (nginx)
    Tasks: 3 (limit: 8639)
   Memory: 11.4M
      CPU: 5.184s
   CGroup: /system.slice/nginx.service
           └─64269 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
           └─64270 "nginx: worker process"
           └─64271 "nginx: worker process"

Dec 08 05:25:06 ecs-c54c systemd[1]: Starting A high performance web server and a reverse proxy server...
Dec 08 05:25:06 ecs-c54c systemd[1]: Started A high performance web server and a reverse proxy server.

```

此时在服务器域名下可以查看在服务器上部署的前端页面（本项目的管理员账户）。



4. 生产环境配置(Gunicorn)

在后端的虚拟环境中配置 `gunicorn`：

```
pip install gunicorn
```

然后根据这个配置，在后端文件夹中的 `gunicorn_config.py` 中加入如下代码：

```
# 绑定的地址和端口
bind = "127.0.0.1:5000"
# 工作进程数
workers = 3
# 访问日志文件
accesslog = "/var/www/CampusFoodExpress/logs/gunicorn_access.log"
# 错误日志文件
errorlog = "/var/www/CampusFoodExpress/logs/gunicorn_error.log"
```

同时由于 `gunicorn` 使用 `wsgi.py` 作为入口，我们需要调整 `/etc/systemd/system/flaskapp.service` 的相关 `service` 配置。修改如下：

```
[Unit]
Description=Gunicorn instance to serve Flask app
After=network.target

[Service]
User=root
Group=www-data
WorkingDirectory=/var/www/CampusFoodExpress
Environment="PATH=/var/www/CampusFoodExpress/.venv/bin"
ExecStart=/var/www/CampusFoodExpress/.venv/bin/uwsgi --http :5000 --gevent 1000 --http-
websockets --master --wsgi-file wsgi.py --callable app
Environment="PYTHONPATH=/var/www/CampusFoodExpress"

[Install]
WantedBy=multi-user.target
```

然后使用如下命令可以重启生产端服务：

```
sudo systemctl daemon-reload
sudo systemctl restart flaskapp
sudo systemctl status flaskapp
```